



Manual AlunosApp

Do Windows Explorer ao APK instalado no celular. Cada passo indica **o que fazer**, **onde clicar** e **o que deve aparecer**. Siga na ordem — não pule nenhum.

Progresso:

15 de 15 passos

1

Criar a pasta alunosapp no Windows Explorer

CONCLUÍDO

Onde: Área de trabalho ou Documentos → botão direito → *Novo* → *Pasta*.

Nomeie a pasta como alunosapp (tudo em minúsculas, sem espaço). Esse nome **não precisa** ser igual ao nome do repositório, mas ajuda se for.

Deve aparecer: uma pasta vazia chamada alunosapp.

✓ Concluído

2

Criar a subpasta www dentro de alunosapp

CONCLUÍDO

Abra a pasta alunosapp e crie uma subpasta chamada www — **exatamente com esse nome**. O Capacitor exige essa nomenclatura.

alunosapp/ └─ **www/** (recém-criada, vazia)

⚠ Não use WWW, web, public ou src. Só www.

✓ Concluído

3

Criar os 5 arquivos dentro de www

CONCLUÍDO

conteúdo nos próximos passos.

```
alunosapp/ └─ www/ └─ index.html └─ style.css └─ app.js └─ manifest.json └─ sw.js
```

⚠ Se o Windows esconder extensões, ative a visualização: *Exibir* → *Mostrar* → *Extensões de nome de arquivo*. Caso contrário o arquivo pode ficar como `index.html.txt`.

✓ Concluído

4 Criar a subpasta icons dentro de www

CONCLUÍDO

Abra `www`, crie uma subpasta chamada `icons`. Ela vai receber os dois PNGs de ícone no próximo passo.

```
alunosapp/ └─ www/ └─ index.html └─ style.css └─ app.js └─ manifest.json └─ sw.js └─ icons/ (vazia)
```

✓ Concluído

5 Colar o código em index.html

CONCLUÍDO

Abra `index.html` no Bloco de Notas (ou VS Code). Clique em  **Copiar** abaixo e cole. Salve.

 **www/index.html**

 **Copiar**

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-sc
  <title>AlunosApp</title>
  <link rel="stylesheet" href="style.css">
  <link rel="manifest" href="manifest.json">
  <meta name="theme-color" content="#0284c7">
  <link rel="icon" type="image/png" sizes="192x192" href="icons/icon-192.png">
</head>
<body>
  <div class="container">
```

```
<div>
  <h1>AlunosApp</h1>
  <p>Gestão Escolar</p>
  ...

```

✓ Concluído

6 Colar o código em style.css

CONCLUÍDO

www/style.css

Copiar

```
.topo {
  display: flex;
  align-items: center;
  gap: 12px;
  padding: 6px 0;
}
.icone {
  width: 44px;
  height: 44px;
  background: linear-gradient(135deg, #0284c7, #38bdf8);
  border-radius: 10px;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 1.4rem;
}
.topo h1 { font-size: 1.3rem; color: #f8fafc; }
.topo p { font-size: 0.75rem; color: #94a3b8; }
```

✓ Concluído

7 Colar o código em app.js

CONCLUÍDO

⚠ Use o botão **Copiar** — não selecione o texto manualmente. O código usa caracteres que se quebram ao copiar à mão.

```
tbody.innerHTML = '';  
if (alunos.length === 0) {  
    tbody.innerHTML = '<tr><td colspan="4" style="text-align:center;color:#64748f">Nenhum aluno cadastrado.</td></tr>';  
    return;  
}  
for (let i = 0; i < alunos.length; i++) {  
    const a = alunos[i];  
    const tr = document.createElement("tr");  
  
    const tdId = document.createElement("td");  
    tdId.textContent = "#" + a.id;  
  
    const tdNome = document.createElement("td");  
    const strong = document.createElement("strong");  
    strong.textContent = a.nome;  
    tdNome.appendChild(strong);  
  
    const tdIdade = document.createElement("td");  
    tdIdade.textContent = a.idade ? a.idade + " anos" : "-";  
    tr.appendChild(tdId);  
    tr.appendChild(tdNome);  
    tr.appendChild(tdIdade);  
    tbody.appendChild(tr);  
}
```

✓ Concluído

8 Colar o código em manifest.json

CONCLUÍDO

www/manifest.json

Copiar

```
{  
  "name": "AlunosApp - Gestão Escolar",  
  "short_name": "AlunosApp",  
  "description": "App de cadastro de alunos com LocalStorage",  
  "start_url": "./index.html",  
  "scope": "./",  
  "display": "standalone",  
  "orientation": "portrait",  
  "background_color": "#0f172a",  
  "theme_color": "#0284c7",  
  "icons": [  
    {  
      "src": "icons/icon-192.png",  
      "sizes": "192x192",  
      "type": "image/png",  
      "purpose": "any"  
    },  
    {  
      "src": "icons/icon-512.png",  
      "sizes": "512x512",  
      "type": "image/png",  
      "purpose": "any"  
    }  
  ]  
}
```

 Concluído

9 Colar o código em sw.js

CONCLUÍDO

 www/sw.js Copiar

```
const CACHE = 'alunosapp-v1';
const ASSETS = [
  './',
  './index.html',
  './style.css',
  './app.js',
  './manifest.json',
  './icons/icon-192.png',
  './icons/icon-512.png'
];

self.addEventListener('install', e => {
  e.waitUntil(caches.open(CACHE).then(c => c.addAll(ASSETS)));
  self.skipWaiting();
});

self.addEventListener('activate', e => {
  e.waitUntil(
    caches.keys().then(keys =>
      Promise.all(keys.filter(k => k !== CACHE).map(k => caches.delete(k)))
    )
  );
});
```

⚠ Toda vez que você editar o app no futuro, troque alunosapp-v1 por alunosapp-v2, -v3... Isso força o celular a baixar a versão nova.

 Concluído

10 Baixar os dois ícones PNG

CONCLUÍDO

Clique nos botões abaixo. Cada um gera e baixa um arquivo PNG na pasta Downloads do seu computador.

**icon-192.png**

192 × 192 px

[Baixar 192×192](#)**icon-512.png**

512 × 512 px

[Baixar 512×512](#)

Depois de baixar: mova os dois arquivos da pasta *Downloads* para `alunosapp\www\icons\`.

Deve aparecer: `icon-192.png` e `icon-512.png` dentro de `www/icons/`.

✓ Concluído

11 Conferir a estrutura final da pasta local

CONCLUÍDO

Antes de subir para o GitHub, confira se a estrutura está **exatamente** assim:

```
alunosapp/ ├── www/      ├── index.html  ├── style.css  ├── app.js    ├──  
manifest.json ├── sw.js    ├── icons/    ├── icon-192.png  
icon-512.png
```

Não deve existir: `www/www/`, arquivos `.txt` extras, ou `index.html` fora de `www/`.

✓ Concluído

12 Criar o repositório alunosapp no GitHub

CONCLUÍDO

Onde: `github.com` → botão **+** no canto superior direito → *New repository*.

Preencha:

- **Repository name:** `alunosapp` (sem espaço, sem acento)
- **Description:** opcional
- **Public** ✓ (obrigatório para usar Pages e Actions gratuitos)
- **Add a README file:** deixe **desmarcado**

Deve aparecer: a página do repositório com instruções de "...or push an existing repository".

✓ Concluído

13 Subir a pasta alunosapp para o repositório

CONCLUÍDO

Você pode fazer isso pelo próprio navegador, sem instalar Git. O GitHub preserva automaticamente a estrutura de pastas do que você arrastar.

Onde: página do repositório → botão **Add file** → *Upload files*.

1. Abra a pasta local alunosapp (a pasta **de fora**, que contém a subpasta `www`).
2. Selecione **a subpasta `www` inteira**.
3. Arraste `www` para a área de upload do GitHub.
4. No campo **Commit changes**, deixe a mensagem padrão.
5. Clique em **Commit changes** (botão verde).

⚠ Arraste a **pasta `www`**, não os arquivos soltos. O GitHub recria a estrutura `www/` automaticamente — inclusive a subpasta `icons/` com os PNGs.

Deve aparecer: no repositório, a pasta `www/` com os 5 arquivos e a subpasta `icons/` com os 2 PNGs.

✓ Concluído

14 Criar o workflow `.github/workflows/build.yml`

CONCLUÍDO

Onde: repositório → **Add file** → *Create new file*.

No campo de nome do arquivo, digite **exatamente** (com barras):

```
.github/workflows/build.yml
```

Ao digitar as barras, o GitHub cria as pastas automaticamente. O campo deve mostrar o breadcrumb `.github > workflows > build.yml`.

 **.github/workflows/build.yml** **Copiar**

```
cat > capacitor.config.json <<'EOF'
{
  "appId": "com.almeida.alunosapp",
  "appName": "AlunosApp",
  "webDir": "www",
  "bundledWebRuntime": false
}
EOF

- name: Verificar conteúdo de www/
  run: ls -la www/

- name: Adicionar plataforma Android
  run: npx cap add android

- name: Sincronizar Capacitor
  run: npx cap sync android

- name: Compilar APK
```

 **Concluído**

15 Ativar o GitHub Pages e testar

CONCLUÍDO

15.1. Ativar Pages

Onde: repositório → **Settings** → menu lateral *Pages*.

1. Em **Build and deployment** → **Source**, mude para **GitHub Actions** (não "Deploy from a branch").
2. Não precisa clicar em Save.

15.2. Rodar o workflow

Onde: aba **Actions** → coluna esquerda *Deploy PWA & Build APK* → botão **Run workflow** (canto direito).

15.3. Confirmar sucesso

Depois de ~2 minutos, dois jobs devem ficar verdes:

- **build-apk** ✓ — gera o APK.

Em **Settings** → **Pages** deve aparecer a mensagem verde:

✓ "Your site was last deployed to the github-pages environment by the Deploy PWA & Build APK workflow."

15.4. Testar o PWA no navegador

Abra no celular (Chrome) ou no PC:

<https://almeida-cma.github.io/alunosapp/>

Você deve ver o AlunosApp, conseguir cadastrar aluno e apagar. No Chrome Android, aparecerá o aviso **"Adicionar à tela inicial"**.

15.5. Baixar o APK e mandar para o celular

1. Vá em **Releases** → **Draft a new release**.
2. Em *Choose a tag*, digite `v1.0.0` → *Create new tag*.
3. Clique em **Publish release**.
4. Aguarde ~2 minutos. O APK `app-debug.apk` aparece anexado na Release.
5. Baixe o APK. Envie para o celular por WhatsApp, USB ou e-mail.
6. No celular, abra o arquivo. Autorize *fontes desconhecidas* se pedir. Instale.
7. Pronto — ícone **AlunosApp** na gaveta de apps.

Deve aparecer: o app abre em tela cheia, sem barra de navegação, com o CRUD funcionando igual ao site.

🔔 **Se o CRUD mostrar `#{a.id}` ou `#{a.nome}`:** o APK contém uma versão antiga do `app.js` (gerada antes da correção). **Como resolver:** desinstale o AlunosApp do celular, confirme que o `www/app.js` no GitHub é a versão corrigida (com `createElement`), rode o workflow novamente em *Actions* → *Run workflow*, baixe o novo APK pela Release e reinstale.

✓ Concluído



Manual concluído!

Você criou do zero um app Web que:

- ☒ Foi empacotado como APK Android.
- ☒ Está publicado em usuario.github.io/alunosapp/ .
- ☒ Tem uma Release em github.com/usuario/alunosapp/releases .

Para atualizar o app no futuro: edite os arquivos em [www/](#) , troque a versão do [sw.js](#) (v1 → v2), faça commit e rode o workflow.